

Overview

Track 16.1 — 🐍 Python for AI Engineering · 16-ai-tech-stack/tracks/python/README.md

🐍 Track 16.1 — Python for AI Engineering

Where you are: Module 16 (AI Tech Stack), track 1 of 7 — the language everything else in this repo is written in. **Time:** ~15–18 hours · **Prereq:** you can already program. Any language. You do **not** need to know Python idioms.

Every other track in this module — LangChain, LangGraph, LangSmith, Bedrock, MCP servers, Claude Code hooks — assumes you can write Python that a colleague would approve. Not Python that runs; Python that is typed, tested, packaged, async where it matters, and vectorized where it counts. That gap is the single most common reason a promising prototype never becomes a service.

This track is deliberately *not* "learn Python". It is the ~20% of modern Python that AI engineering actually leans on, with the 2026 toolchain (uv, ruff, pydantic v2, free-threaded builds) rather than the 2018 one (pip, venv, setup.py, `%timeit`).

Learning objectives

By the end of this track you can:

1. Stand up a reproducible AI project with `uv`, a single `pyproject.toml`, `ruff`, a type checker and `pytest` — in under two minutes, from memory.
2. Explain why a Python loop is ~1000× slower than a BLAS call, and rewrite numeric code to vectorize — while predicting the memory cost of the broadcast you just introduced.
3. Write async code that batches LLM calls with `asyncio.gather`, bounds concurrency with a `Semaphore`, and retries with exponential backoff **plus jitter** — and say precisely why jitter is not optional.
4. State what the GIL does and does not prevent, pick correctly between `asyncio` / `threads` / `processes` for a given workload, and describe the actual status of free-threaded CPython.
5. Model LLM structured output with Pydantic v2: generate the JSON Schema you send the provider, validate what comes back, and drive a bounded repair loop on failure.
6. Lay out, configure, log, test and profile an AI service — including mocking an LLM client and writing tests that survive nondeterminism.
7. Recognise on sight the six Python idioms that reliably bite AI engineers, and say what each one costs.

Suggested path

#	Do this	File	Time
1	Read the core concepts	notes/01-modern-python-toolchain.md	3h
2	Read the practitioner craft	notes/02-numeric-async-and-performance.md	3h
3	Run and break the benchmark	code/vectorization_benchmark.py	1h
4	Run the async client demo	code/async_llm_client.py	1.5h
5	Run the structured-output demo	code/pydantic_structured_output.py	1.5h
6	Do the exercises	lab/EXERCISES.md	6h
7	Skim the primary sources	papers/PAPERS.md	1.5h
8	Present it back	slides/ (python.pptx)	0.5h

Run everything with the repo venv so you are not fighting your system Python:

```
/Users/srinip/ai-all/.venv/bin/python 16-ai-tech-stack/tracks/python/code/vectorization_benchmark.py
```

All three scripts are **offline**: no API keys, no network, no cost.

The 18 terms you must own

uv · pyproject.toml · lockfile · ruff · type hint · Pydantic v2 · TypedDict · dataclass · broadcasting · vectorization · dtype · GIL · free-threaded build · coroutine · event loop · semaphore · exponential backoff + jitter · generator

The one-command project setup

Memorise this. It replaces pip, venv, virtualenv, pyenv, pip-tools and poetry:

```
uv init my-ai-service && cd my-ai-service
uv add anthropic pydantic httpx
uv add --dev pytest pytest-asyncio ruff mypy
uv run pytest                # creates the venv, resolves, locks, and runs – one command
```

Exit check

You can hand a colleague a repo that: installs with one `uv sync`, passes `ruff check` and `mypy` clean, has a `pytest` suite that mocks an LLM client and runs in under five seconds with no network, exposes a Pydantic-validated structured-output path with a repair loop, and batches its model calls concurrently under an explicit rate limit. If they can clone it and get a green test run before their coffee cools, you passed.